T01.S01 — The Discipline Map & Anatomy of a Game

Topic: T01 — Real-Time Programming Foundations | **Depends on:** — **Status:** expanded | **Assignments:** — | **Est. theory time:** 1–2 sessions

> The question this subtopic answers: *what is actually in a game, who builds each part, and what happens between the moment you press a key and the moment the screen changes?* Everything else in the roadmap is a zoom-in on one box of the diagram you build here.

Theory

U01 — What a game program is, structurally

What it is. A game is a program with one dominant loop that, forever: reads input, advances a simulation by some amount of time, and produces one image and one audio buffer. That's it. Everything else is detail hanging off those three verbs.

Why it exists. Most software is *reactive* — it sits idle until something happens. A game is *proactive*: it runs continuously whether or not you touch it, because the world has to keep existing. This single difference reorganises everything: there is no "wait for input", there is "what is the state of the input right now, at this instant in the loop".

The mechanism. One frame, expanded:

```
+-----------------------------------------------------------+
| 1. Poll OS events            (window, keyboard, mouse, pad) |
| 2. Build input state          (current, previous, deltas)   |
| 3. Fixed-step simulation      (repeat while time remains)   |
|      - gameplay update / scripts / AI                       |
|      - physics step                                         |
|      - animation sample                                     |
| 4. Late update                (cameras, attachments)        |
| 5. Cull + build render lists  (what is visible)             |
| 6. Submit draw calls to GPU                                 |
| 7. Update audio (mix runs on its own thread)                |
| 8. Present / swap buffers     (may block on vsync)          |
+-----------------------------------------------------------+
                    repeat, ~16.67 ms later
```

Two properties of that list matter more than the list itself:

1. **The order is a dependency chain, not a preference.** Rendering must come after simulation because it draws simulation results. Culling must come after camera update because visibility depends on the camera. When you see an engine reorder these, there is a reason.
2. **Not every box runs at the same rate.** Step 3 may run zero, one, or four times in a frame (that's `T03.S01`). Step 7 runs on a separate thread at the audio device's rate (that's `T11.S01`). The frame is the *rendering* rate, and conflating it with the simulation rate is the single most common beginner architecture error.

Where it bites. Engines that expose only "Update()" hide step 3's multiplicity from you, and then your movement code is framerate-dependent and you don't know why. The `Update()` vs `FixedUpdate()` split in Unity is exactly this distinction, surfaced.

Read.

- `[GEA3]` ch. 1.5–1.7 — engine subsystems and the runtime architecture diagram. Look at the layer diagram in 1.6 specifically; you will come back to it every phase.
- `[GPP]` "Game Loop" — https://gameprogrammingpatterns.com/game-loop.html — read the whole chapter now, and again at `T03.S01`.

Check yourself.

1. Why can't rendering and simulation simply run at the same rate, always?
2. Name two boxes in the frame diagram that could be swapped without breaking anything, and two that could not.
3. Where does audio sit in this picture, and why is that answer different from every other subsystem?

U02 — The subsystem map of an engine

What it is. The standard decomposition of a game engine into layers, each depending only on those below it.

Why it exists. Because "a game" is 200k+ lines of unrelated concerns, and without a layering rule they become mutually dependent within a year. The layering is what lets a rendering change not break audio.

The mechanism. Bottom to top, roughly `[GEA3]` ch. 1.6's structure:

```
Game-specific       | game modes, the actual game's rules and content
------------------- |
Gameplay foundation | entities/ECS, events, scripting, world/scene
------------------- |
Runtime systems     | renderer | physics | animation | audio | AI | net
------------------- |
Resources           | asset handles, loading, streaming
------------------- |
Core                | math, containers, allocators, strings, time, jobs
------------------- |
Platform            | window, input devices, filesystem, threads, GPU API
```

Map this roadmap onto that stack and the ordering stops looking arbitrary: `T02` and `T03` build the Core layer; `T03.S05` builds Resources; `T04`, `T05`, `T08`–`T11`, `T14` build Runtime Systems; `T06` builds Gameplay Foundation; `T07` and `T16` are game-specific.

Where it bites. The layering is violated first, and most quietly, by "just one" upward reference — a core math type that knows about a rendering enum, a physics body that holds a pointer to a game entity. Each one is fine; the twentieth means you cannot build the physics tests without linking the renderer.

Read.

- `[GEA3]` ch. 1.6 — the runtime engine architecture. The single most useful diagram in the book.
- Browse a real engine's top-level directory listing (Godot's `servers/`, `scene/`, `core/` split is unusually legible) and try to place each directory in the stack above.

Check yourself.

1. Which layer does a `Transform` type belong to, and why is that answer contested?
2. Give a concrete example of an upward dependency that would be hard to detect and expensive to remove.

U03 — Who does what: the disciplines

What it is. "Game developer" is about a dozen distinct jobs. Knowing which one you are aiming at changes what you emphasise for the next two years — though not, importantly, the foundations.

Why it exists. Team sizes and specialization. On a 200-person team these are separate people; on a 3-person team one person does five of them; either way the *skills* decompose the same way.

The mechanism. The technical roles, with what the day actually contains:

role	daily work	this roadmap's coverage
Gameplay programmer	Implements mechanics, tunes feel, works closest to designers. Most numerous role.	T05.S05, T06, T07
Engine/systems programmer	Core, memory, threading, the loop, platform layers.	T03, T13
Graphics programmer	Renderer, shaders, GPU performance. Smallest and most competitive.	T04, T08
Physics programmer	Solvers, collision, character movement. Often merged with engine.	T05
Animation/tech-animation programmer	Rigs, blend systems, IK, the animation-gameplay boundary.	T09
AI programmer	Behaviour, pathfinding, perception, director systems.	T10
Tools programmer	Editors, pipelines, iteration speed. Underrated; many open roles.	T12
Network programmer	Replication, prediction, backend integration. Scarce and well paid.	T14
Audio programmer	Mixing, spatialization, middleware integration. Very scarce.	T11
Technical artist / designer	The bridge role: shaders and tooling for artists, or systems for designers.	T08, T12, T07

Non-programming disciplines you will work with constantly and should be able to talk to: designers (systems, level, narrative), artists (concept, 3D, animation, VFX, UI), audio (composer, sound designer), production, and QA.

Where it bites. Choosing too early. Someone who decides at month two that they are a graphics programmer skips `T05` – `T07` and becomes an engineer who can render beautifully and has never shipped a game. `T15.S01` is where this roadmap asks you to choose, deliberately late.

Read.

- `[GEA3]` ch. 2.1 and the introduction's discussion of team structure.
- Read five actual job postings for three different roles above. Note what is *required* vs *nice to have* — the gap between them is very informative.

Check yourself.

1. Which two roles have the most overlap with the C++ backend/systems experience you already have?
2. Which role would you enjoy on a bad day, not a good one?

U04 — What makes games different as software

What it is. The properties that make game code diverge from the server/application code you have been writing for a year.

Why it exists. Worth making explicit, because a lot of instincts that serve you well elsewhere are actively counterproductive here — and being surprised by that repeatedly is worse than being told once.

The mechanism. The five that actually change your decisions:

1. **Soft real-time deadline.** 16.67 ms, every frame. Not "fast" — *on time*. A 300 ms GC pause is not slow, it is a visible failure. Consequence: predictable performance beats peak performance; allocation patterns matter more than allocation speed.
2. **Simulation, not transaction.** State is continuously mutating and massively interconnected. There is no "request" to isolate. This is why the ECS/data-layout question (`T03.S04`) even exists.
3. **Requirements are discovered, not specified.** "Is the jump fun?" is answered by building it and playing it. Consequence: iteration speed is a top-three engineering priority (`T12.S04`), and over-abstraction is punished harder than duplication.
4. **Correctness is perceptual.** The bar is "does the player notice", not "is it correct". Physics is deliberately inaccurate; AI deliberately handicapped; animation deliberately not what the physics says. This feels wrong for about six months and then feels obvious.
5. **The product is one large mutable artifact.** Content and code ship together as one build; there is no rolling deploy and often no patch. Consequence: `T12` and `T16` matter, and binary asset workflows break normal git habits (`T12.S05`).

Where it bites. Point 3 catches experienced engineers hardest. Applying a "design it properly first" instinct to gameplay code produces elaborate systems for mechanics that get cut two weeks later. The correct instinct is: prototype ugly, then rewrite once the design is known.

Read.

- `[GPP]` "Architecture, Performance, and Games" — https://gameprogrammingpatterns.com/architecture-performance-and-games.html — this exact tension, argued well. Short, essential.
- `[MIKEACTON]` — CppCon 2014 "Data-Oriented Design and C++". Watch it now for the worldview; you will apply it in `T03.S04` and `T13.S03`.

Check yourself.

1. Give an example from your own last year of code where "design it properly first" was right, and explain what makes gameplay code different.
2. What does "soft" real-time mean, and which parts of a game are *hard* real-time?

Implementation

No assignment attached — this subtopic is orientation.

But do one 20-minute exercise, and keep the output:

Write the frame diagram from U01 for a game you know well. Pick something you have played 50+ hours of. For each box, write what that specific game must be doing there. Where you can't fill a box in, that is a topic you will meet later — note which `TNN` it belongs to.

Keep it in `game-development/code/notes-T01S01-frame-map.md`. You will re-read it at `A023` and again at `T15.S01`, and both times it will be usefully wrong.

Discussion Log

Open Questions

Connections

- `T01.S02` — the timing constraint that the frame diagram runs under.
- `T03.S01` — box 3 of the frame diagram, in full detail.
- `T15.S01` — the role choice this unit only introduces.